

設備管理與維護系統 Final Project Part III

Final Project Part III > [GitHub Repository: 114-2-DB_System-Project](#)

目錄

- 一、應用情境
- 二、使用案例
- 三、系統需求說明 (Functional Requirements)
- 四、完整性限制 (Integrity Constraints)
- 五、ER Diagram (Entity-Relationship Diagram)
- 六、Database Schema / SQL Schema
- 七、View 設計與使用說明

一、應用情境

- 背景：**大學系所轄下的空間（含系辦公室、教學實驗室、教授專屬實驗室與一般教室）設備與物品種類繁多，依管理方式可分為三類：
 - 財產設備：**貼有學校或系所財產標籤列管（如：伺服器、高階儀器、投影機）。
 - 可重複使用的非列管設備：**未貼財產標籤管理（如：簡報筆、轉接頭、開發板、系辦推車）。
 - 日常耗材：**單價較低、會被頻繁消耗（如：白板筆、影印紙、教學用電子零件）。
- 痛點：**
 - 資產流向不明與跨空間調度困難：**高價設備與共用硬體若僅靠紙本或各實驗室自行記憶管理，常因教職員離職、學生畢業或設備在不同教室間借用而導致流向不明。當設備損壞時，缺乏跨空間的「使用歷程追蹤」，無法追溯上一個借用者或釐清是教學損耗還是人為破壞，維修成本往往只能由系辦公室吸收。
 - 耗材領用成行政瓶頸：**系上耗材（尤其是教學實驗耗材或辦公文具）的領用極度依賴系助教或行政人員人工登記。全系師生頻繁的領用需求嚴重消耗行政時間，且常發生「先拿了再說、事後忘記補登記」的狀況，導致系辦系統帳面庫存與實際數量嚴重脫節，影響後續採購編列。
- 解決方案：**本專題擬開發一套「設備管理與維護系統(以系所為例)」。
 - 針對具名資產與共用設備：**系統將建立完整的「使用歷程追蹤 (Audit Trail)」機制，精準記錄每一次在全系不同空間的借還時間與經手人員；當報修時，系管人員可一鍵查詢該設備的歷史流向，釐清責任歸屬。
 - 針對耗材：**系統導入「自助領用與審計機制」，將登記責任分散給全系使用者，並結合低水位預警功能，大幅降低系辦的行政負擔，確保系所資源透明與妥善分配。

二、使用案例

角色一：系所管理員 (Department Administrator / 系助教)

UC1：建立資產基本資訊 (Create Basic Asset Information)

- 目標：**建立全系設備或耗材的初始數位身分。

- **描述：**系辦取得新物品後，建立獨立的 **內部設備編號**，記錄名稱、類型、狀態與歸屬空間（如：系辦公室或特定實驗室）。若為 **財產設備**，需額外登錄學校財產編號、經費來源、保管人（通常為教授或系主任）、取得日期、金額與耐用年限。

關鍵規則： 內部設備編號 不可重複；若物品類型為 **財產設備**，**財產編號** 不得為空且不可重複。

UC2：耗材庫存管理 (Consumable Stock Management)

- **目標：**追蹤全系公用或教學耗材庫存與採購需求。
- **描述：**管理員建立耗材資料，記錄目前庫存量、最低庫存量與單位。師生領用後扣除庫存；低於最低庫存量時，系統提醒系辦進行採購。

關鍵規則： 耗材庫存不可 < 0 ，領用數量不可 $>$ 目前庫存量。

UC3：設備停用與報廢狀態更新 (Equipment Deactivation and Disposal Status Update)

- **目標：**記錄系所設備生命週期後期的狀態變更。
- **描述：**當設備超過耐用年限或不堪使用時，系所管理員可將狀態更新為 **停用** 或啟動 **報廢** 流程，並記錄異動時間、操作者與原因，以利後續向學校總務處核銷。

關鍵規則： 更新為 **報廢** 時須產生狀態異動紀錄；已報廢物品不得再借用或建立維修工單。

角色二：全系師生 (Faculty and Students)

UC4：設備借用與歸還 (Equipment Borrowing and Return)

- **目標：**追蹤共用設備的實體位置與責任人。
- **描述：**師生查詢系上可借用設備後發起借用申請，系統記錄借用時間、預計歸還時間、實際歸還時間與借用狀態。

關鍵規則： 耗材 不得建立借用紀錄，僅能透過領用紀錄管理。

UC5：故障異常報修 (Issue Reporting)

- **目標：**快速反應教學或研究設備的失效狀態。
- **描述：**師生於教室或實驗室發現設備異常，建立維修工單通報系辦或設備負責人。

角色三：空間/設備負責人 (Space/Equipment Supervisor)

(各實驗室專責助教或工讀生)

UC6：維修結案與回報 (Maintenance Fulfillment)

- **目標：**紀錄各空間設備的維修成本與結果。
- **描述：**設備完成維護後，負責人回填維修金額、廠商資訊與更換紀錄。

UC7：預防性維護排程 (Maintenance Scheduling)

- **目標：**確保系上精密儀器或重要設施正常運作。
- **描述：**系統依據保養週期自動通知該空間/設備負責人進行檢修。

三、系統需求說明 (Functional Requirements)

1. 功能性需求 (Functional Requirements)

A. 物品分類與內部識別資訊管理

- 物品分類管理：支援 財產設備、可重複使用的非列管設備 與 耗材 三類。
- 系所內部設備資訊建立：每項受管設備須有獨立的 內部設備編號，作為跨空間查詢與追蹤依據。
- 基本資料管理：記錄名稱、類別、存放空間 (特定教室/實驗室/系辦)、空間負責人、狀態與保固。
- 規格與數量管理：同規格設備可依管理需求以個別物品編號追蹤，或於非列管設備資料中記錄數量。

B. 財產設備管理

- 財產標籤資料登錄：記錄學校財產編號、保管人 (教職員)、經費來源、取得金額與耐用年限。
- 保管人與設備負責人區分：保管人 為財產層級負責人 (教授)；設備負責人 為實際管理日常運作的人員 (助教或研究生)。
- 報廢評估：提供報廢建議供系辦參考，以銜接校方總務系統的報廢流程。

C. 可重複使用的非列管設備管理

- 無財產標籤設備管理：針對簡報筆、轉接線等，以系所內部編號管理。
- 借用狀態管理：保留跨系所空間借用的使用者、時間與歸還情形。

D. 耗材庫存與自助領用

- 耗材資料管理：支援影印紙、實驗材料等管理，記錄存放位置 (如系辦物資櫃)。
- 自助領用登記：師生領用時記錄身分、數量與用途，即時扣除庫存。
- 低庫存預警：自動提醒系辦行政人員補貨。

E. 使用歷程與審計追蹤

- 借還與責任溯源：完整記錄全系設備借還，發生遺失損壞時可追溯上一位經手的師生。
- 狀態異動紀錄：記錄 可用、借出、維修、停用、報廢 等變更歷程。

F. 維修工單與維護管理

- 跨空間故障回報：師生依編號建立故障回報。
- 維修與成本統計：記錄廠商、費用，累計設備維護成本供系所經費編列參考。

2. 非功能性需求 (Non-Functional Requirements)

A. 資料一致性與完整性

- 系所內部編號 不可重複；財產編號 不可為空且不可重複。
- 庫存不可 < 0 ；領用量不可 $>$ 庫存。
- 維修結案與歸還日期的時序邏輯必須正確。

B. 權限控管 (Role-Based Access Control)

- 全系師生：可查詢、借用、領用、報修。
- 空間/設備負責人：管理轄下空間設備狀態、處理報修工單。
- 系所管理員：擁有全系最高權限，可進行跨空間盤點、報廢審查、庫存採購與帳號權限管理。

C. 可追溯性 與 D. 庫存資料一致性

- 確保高併發下（如開學初大量領用耗材）的庫存扣減正確性。
- 歷史紀錄採封存 (Archived) 而非物理刪除。

四、完整性限制 (Integrity Constraints)

1. 實體完整性 (Entity Integrity)

每一項物品必須具備獨立的 內部設備編號 (Primary Key)，且不得缺失 (Not Null)。歷程紀錄亦須具備獨立識別資訊。

2. 參照完整性 (Referential Integrity)

歸屬之 類別、存放地點（如特定教室編號）及 負責人 (Foreign Keys) 必須對應系統中有效實體。借用與維修關聯之設備必須為有效實體。

3. 值域完整性 (Domain Integrity)

- 狀態 (Status)：限於 可用、借出中、維修中、停用、報廢 或 遺失。
- 數值 (Numeric)：財務與庫存數據須為非負值 (≥ 0)。
- 日期 (Date)：須符合標準 Timestamp 格式。

4. 使用者自訂完整性 (User-defined Integrity)（許願池）

- 財產資料與維修處理限制：財產設備必填 財產編號 與 保管人。維修工單建立後可指派設備負責人處理。
- 借用與領用限制：耗材不可借用，僅能領用。異常狀態設備不可借用。單一設備同一時間僅限一筆未歸還紀錄。
- 時序與歷程限制：嚴格控管維修與借還的時間先後順序；有歷史紀錄的物品禁止物理刪除。

5. ■【實體】屬性與限制對應表

A. 物品與存放位置

標示	實體	目標屬性	限制類型	完整性限制 / Business Rules
【實體】	物品	內部唯一編號	實體完整性	不可為空，且不可重複，作為物品主要識別依據
【實體】	物品	名稱	值域完整性	不可為空
【實體】	物品	狀態	值域完整性	僅能為 可用、借出中、維修中、停用、報廢、遺失
【實體】	物品	保固期限	值域完整性	必須為合法日期格式
【實體】	存放位置	空間編號	實體完整性	不可為空，且應能唯一識別存放位置
【實體】	存放位置	空間名稱	值域完整性	不可為空
【實體】	存放位置	空間類型	值域完整性	僅能為系統定義的空間類型

B. 物品分類子實體

標示	實體	目標屬性	限制類型	完整性限制 / Business Rules
【實體】	財產設備	財產編號	實體完整性	不可為空，且不可重複
【實體】	財產設備	保管人	使用者自訂完整性	財產設備必須記錄保管人
【實體】	財產設備	取得日期	值域完整性	必須為合法日期格式
【實體】	財產設備	經費來源	值域完整性	應記錄有效經費來源
【實體】	財產設備	取得金額	數值限制	財產設備取得金額必須大於等於 3,000 元
【實體】	財產設備	耐用年限	數值限制	必須為正整數或大於 0
【實體】	可重複使用的非列管設備	規格	值域完整性	不可為空，需描述設備規格
【實體】	可重複使用的非列管設備	數量	數值限制	必須為整數，且大於等於 0
【實體】	可重複使用的非列管設備	可借用	值域完整性	僅能為 是 / 否
【實體】	可重複使用的非列管設備	需歸還	值域完整性	僅能為 是 / 否
【實體】	耗材	庫存量	數值限制	必須為整數，且大於等於 0
【實體】	耗材	最低庫存量	數值限制	必須為整數，且大於等於 0
【實體】	耗材	單位	值域完整性	不可為空，例如：包、盒、支、張

C. 使用者與角色

標示	實體	目標屬性	限制類型	完整性限制 / Business Rules
【實體】	角色	角色編號	實體完整性	不可為空，且不可重複
【實體】	角色	角色名稱	值域完整性	不可為空
【實體】	使用者	使用者編號	實體完整性	不可為空，且不可重複
【實體】	使用者	使用者姓名	值域完整性	不可為空
【實體】	使用者	聯絡方式	值域完整性	應記錄有效聯絡方式
【實體】	系所管理員	無額外屬性	ISA 限制	繼承使用者之使用者編號、使用者姓名、聯絡方式
【實體】	全系師生	無額外屬性	ISA 限制	繼承使用者之使用者編號、使用者姓名、聯絡方式
【實體】	設備負責人	無額外屬性	ISA 限制	繼承使用者之使用者編號、使用者姓名、聯絡方式

D. 狀態異動紀錄

標示	實體	目標屬性	限制類型	完整性限制 / Business Rules
【實體】	狀態異動紀錄	異動編號	實體完整性	不可為空，且不可重複
【實體】	狀態異動紀錄	舊狀態	值域完整性	必須為物品狀態的合法值
【實體】	狀態異動紀錄	新狀態	值域完整性	必須為物品狀態的合法值
【實體】	狀態異動紀錄	異動時間	值域完整性	不可為空，必須為合法時間格式
【實體】	狀態異動紀錄	異動原因	使用者自訂完整性	狀態變更時應記錄異動原因

E. 借用紀錄與領用紀錄

標示	實體	目標屬性	限制類型	完整性限制 / Business Rules
【實體】	借用紀錄	借用紀錄單號	實體完整性	不可為空，且不可重複
【實體】	借用紀錄	借用時間	時序限制	不可為空，且必須早於實際歸還時間
【實體】	借用紀錄	預計歸還時間	時序限制	應晚於借用時間
【實體】	借用紀錄	實際歸還時間	時序限制	若已歸還，不得早於借用時間
【實體】	借用紀錄	借用狀態	值域完整性	僅能為 借用中、已歸還、逾期

標示	實體	目標屬性	限制類型	完整性限制 / Business Rules
【實體】	領用紀錄	領用紀錄單號	實體完整性	不可為空，且不可重複
【實體】	領用紀錄	用途	使用者自訂完整性	領用時應記錄用途
【實體】	領用紀錄	數量	數值限制	必須為正整數，且不可大於對應耗材庫存量
【實體】	領用紀錄	領用時間	值域完整性	不可為空，必須為合法時間格式

F. 維修工單、維修處理紀錄與維護廠商

標示	實體	目標屬性	限制類型	完整性限制 / Business Rules
【實體】	維修工單	維修工單編號	實體完整性	不可為空，且不可重複。
【實體】	維修工單	報修日期	值域完整性	不可為空，必須為合法日期或時間格式。
【實體】	維修工單	故障描述	使用者自訂完整性	報修時必須填寫故障現象，不可為空白內容。
【實體】	維修工單	工單狀態	值域完整性	僅能為 待處理、處理中、已完成、取消。
【實體】	維修處理紀錄	維修處理編號	實體完整性	不可為空，且不可重複。
【實體】	維修處理紀錄	維修費用	數值限制	可為 0，但不得小於 0；若費用尚未確定，應使用 NULL，不可用 0 代表未知。
【實體】	維修處理紀錄	完成日期	值域完整性	必須為合法日期或時間格式。
【實體】	維修處理紀錄	更換零件	使用者自訂完整性	若有更換零件，必須記錄零件名稱、型號或更換內容；未更換時可為 NULL。
【實體】	維修處理紀錄	下次保養日期	時序限制	若有設定，必須晚於該筆維修處理紀錄的完成日期。
【實體】	維修處理紀錄	維修結果	使用者自訂完整性	維修處理完成時必須記錄處理結果，例如已修復、暫時排除、無法修復或建議報廢。
【實體】	維護廠商	維護廠商編號	實體完整性	不可為空，且不可重複。
【實體】	維護廠商	廠商名稱	值域完整性	不可為空；名稱不可為純空白。若未設統一編號，名稱不建議作為唯一識別鍵。
【實體】	維護廠商	廠商聯繫方式	值域完整性	應記錄有效電話或電子郵件等聯絡方式。

6. ♦【關聯】與 △【ISA】限制對應表

標示	關聯 / ISA	連接對象	建議基數	完整性限制 / Business Rules
【關聯】<存放於>	存放於	物品 — 存放位置	N : 1	每項物品必須存放於一個有效的存放位置；一個存放位置可存放多項物品。
【ISA】	物品分類	物品 — 財產設備 / 可重複使用的非列管設備 / 耗材	互斥且完全分類	每項物品必須且只能屬於三種類型之一。
【關聯】<具有>	具有	物品 — 狀態異動紀錄	1 : N	一項物品可有多筆狀態異動紀錄；每筆異動紀錄必須對應一項物品。
【關聯】<執行>	執行	使用者 — 狀態異動紀錄	1 : N	每筆狀態異動紀錄必須由一位使用者執行；一位使用者可執行多筆異動。
【關聯】<建立>	建立	系所管理員 — 物品	1 : N	物品資料由系所管理員建立；一位系所管理員可建立多筆物品資料。
【ISA】	使用者分類	使用者 — 系所管理員 / 全系師生 / 設備負責人	圖中未標明是否互斥	三種角色皆繼承使用者資料；角色必須對應到有效使用者。
【關聯】<管理>	管理	系所管理員 — 借用紀錄	1 : 1	借用紀錄須由系所管理員管理。
【關聯】<申請>	申請	全系師生 — 借用紀錄	1 : N	全系師生可申請多筆借用紀錄；每筆借用紀錄須對應一位申請者。
【關聯】<申請>	申請	全系師生 — 領用紀錄	1 : N	全系師生可申請多筆領用紀錄；每筆領用紀錄須對應一位申請者。
【關聯】<回報>	回報	使用者 — 維修工單	1 : N	一位使用者可回報多張維修工單；每張維修工單必須由一位使用者回報。

標示	關聯 / ISA	連接對象	建議基數	完整性限制 / Business Rules
【關聯】<處理>	處理	設備負責人 — 維修工單	1 : N	一位設備負責人可處理多張維修工單；一張工單在待處理階段可尚未指派處理人，指派後至多對應一位目前處理人。
【關聯】<維修對象>	維修對象	物品 — 維修工單	1 : N	一項設備在生命週期中可產生多張維修工單；每張維修工單必須對應一項有效物品。
【關聯】<產生>	產生	維修工單 — 維修處理紀錄	1 : 0..N	一張維修工單在尚未處理時可沒有處理紀錄；後續可產生多筆處理紀錄。每筆維修處理紀錄只能屬於一張維修工單。
【關聯】<委託>	委託	維護廠商 — 維修處理紀錄	1 : 0..N	一家維護廠商可對應零筆或多筆維修處理紀錄；一筆維修處理紀錄可為系內自行處理，因此 <code>vendor_id</code> 可為 <code>NULL</code> 。
【關聯】<紀錄>	紀錄	財產設備 / 可重複使用的非列管設備 — 借用紀錄	1 : N	可被借用的設備可產生多筆借用紀錄；每筆借用紀錄須對應一項設備。
【關聯】<紀錄>	紀錄	耗材 — 領用紀錄	1 : N	一項耗材可有多筆領用紀錄；每筆領用紀錄必須對應一項耗材。

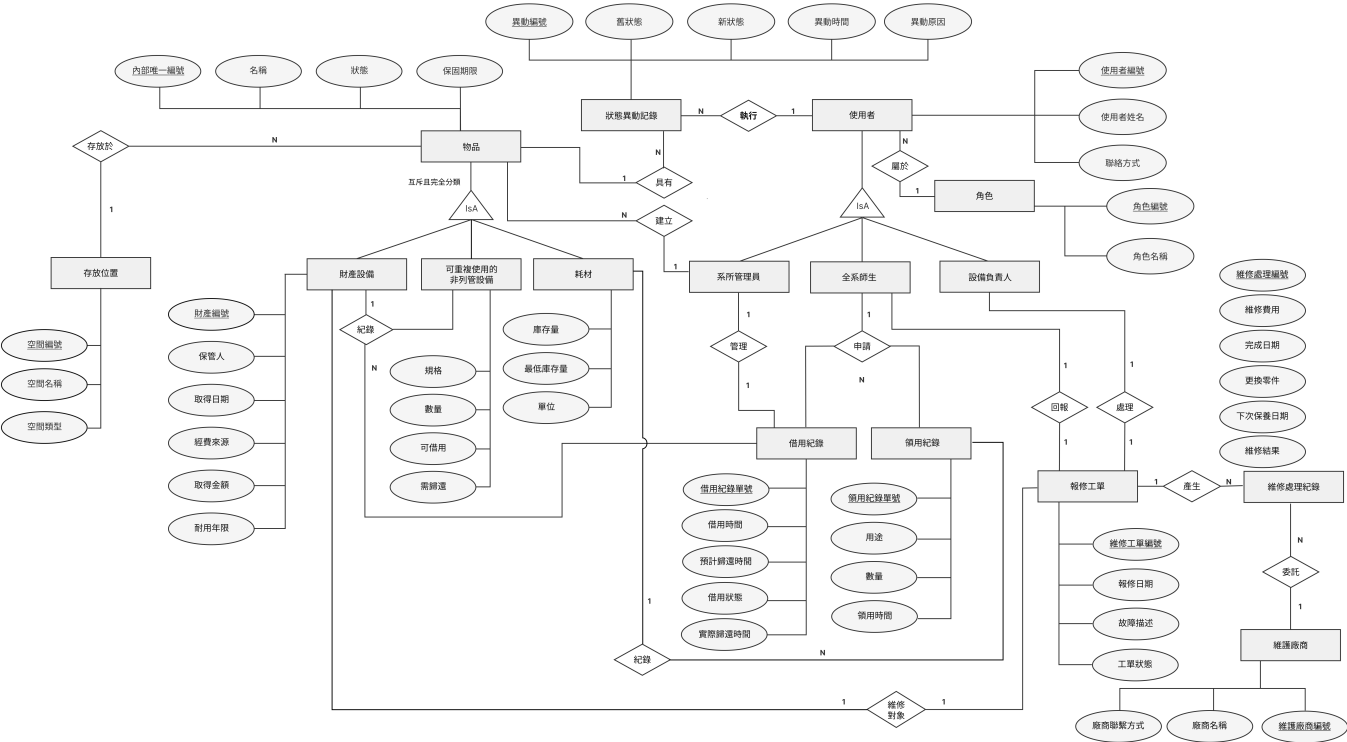
7. 完整性限制對 SQL 設計的影響

限制類型	對應 SQL 設計方式	範例說明
實體完整性	PRIMARY KEY 、 NOT NULL 、 UNIQUE	例如：內部設備編號 不可為空且不可重複
參照完整性	FOREIGN KEY	例如：借用紀錄中的使用者必須存在於使用者資料表
值域完整性	CHECK 、 ENUM 或應用程式檢查	例如：物品狀態只能是 可用 、 借出中 、 維修中 、 停用 、 報廢 、 遺失
數值限制	CHECK	例如：庫存量、維修費用不可小於 0；財產設備取得金額需大於等於 3,000 元
時序限制	CHECK 或 Trigger	例如：實際歸還時間不得早於借用時間
使用者自訂完整性	Trigger 或應用程式邏輯	例如：耗材不可建立借用紀錄；報廢物品不得再借用或維修

五、ER Diagram (Entity-Relationship Diagram) / Database schema

ER Diagram

ER Diagram - 設備管理與維護系統 (以系所為例)



六、Database Schema / SQL Schema

本節整理 13 張 MariaDB 資料表。每一小節先列 Schema 對照表，再列完整 CREATE TABLE 語法，使欄位、鍵值、限制、外鍵規則與索引能逐項對照。

6.1 角色 (role_type)

Schema 對照表

資料型態	欄位名稱	鍵值 / 限制	欄位說明
TINYINT UNSIGNED	role_id	NOT NULL	角色編號。
VARCHAR(5)	role_name	NOT NULL	角色名稱。
表級約束／索引	pk_role_type	PRIMARY KEY	以 role_id 作為角色類型主鍵。
表級約束／索引	uq_role_type_name	UNIQUE	確保 role_name 不重複。
表級約束／索引	ck_role_type_id	CHECK: role_id IN (1, 2, 3)	限制角色代碼只能使用系統定義的三種角色。
表級約束／索引	ck_role_type_name	CHECK: CHAR_LENGTH(TRIM(role_name)) BETWEEN 1 AND 5	避免角色名稱為空白或超出顯示長度。

對應 SQL：CREATE TABLE role_type


```
CREATE TABLE role_type (
    role_id TINYINT UNSIGNED NOT NULL,
    role_name VARCHAR(5) NOT NULL,

    CONSTRAINT pk_role_type PRIMARY KEY (role_id),
    CONSTRAINT uq_role_type_name UNIQUE (role_name),
    CONSTRAINT ck_role_type_id
        CHECK (role_id IN (1, 2, 3)),
    CONSTRAINT ck_role_type_name
        CHECK (CHAR_LENGTH(TRIM(role_name)) BETWEEN 1 AND 5)
) ENGINE = InnoDB;
```

設計重點：

- `role_id` 使用 `TINYINT UNSIGNED`，適合存放固定且少量的角色分類。
- `role_name` 加上 `UNIQUE`，確保同一角色名稱不會重複建立。
- `CHECK` 用於限制角色代碼範圍與名稱長度。

6.2 存放位置 (`space`)

Schema 對照表

資料型態	欄位名稱	鍵值 / 限制	欄位說明
<code>SMALLINT UNSIGNED AUTO_INCREMENT</code>	<code>space_id</code>	<code>NOT NULL, AUTO_INCREMENT</code>	存放空間流水主鍵，支援系所內多個教室、實驗室或辦公室。
<code>CHAR(4) CHARACTER SET ascii COLLATE ascii_bin</code>	<code>space_code</code>	<code>NOT NULL</code>	空間編號。
<code>VARCHAR(30)</code>	<code>space_name</code>	<code>NOT NULL</code>	空間名稱。
<code>TINYINT UNSIGNED</code>	<code>space_type</code>	<code>NOT NULL</code>	空間類型。
表級約束／索引	<code>pk_space</code>	<code>PRIMARY KEY</code>	以 <code>space_id</code> 作為存放空間主鍵。
表級約束／索引	<code>uq_space_code</code>	<code>UNIQUE</code>	確保 <code>space_code</code> 不重複。
表級約束／索引	<code>uq_space_name</code>	<code>UNIQUE</code>	確保 <code>space_name</code> 不重複。
表級約束／索引	<code>ck_space_code</code>	<code>CHECK: space_code REGEXP '^S[0-9]{3}\$'</code>	確保空間代碼符合 <code>S001</code> 這類固定格式。
表級約束／索引	<code>ck_space_name</code>	<code>CHECK: CHAR_LENGTH(TRIM(space_name)) BETWEEN 1 AND 30</code>	避免空間名稱為空白或過長。
表級約束／索引	<code>ck_space_type</code>	<code>CHECK: space_type IN (1, 2, 3, 4)</code>	限制空間類型只能使用定義好的四種代碼。

對應 SQL：`CREATE TABLE space`

```
CREATE TABLE space (
    space_id SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
    space_code CHAR(4) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
    space_name VARCHAR(30) NOT NULL,
    space_type TINYINT UNSIGNED NOT NULL,

    CONSTRAINT pk_space PRIMARY KEY (space_id),
    CONSTRAINT uq_space_code UNIQUE (space_code),
    CONSTRAINT uq_space_name UNIQUE (space_name),

    CONSTRAINT ck_space_code
        CHECK (space_code REGEXP '^S[0-9]{3}$'),
    CONSTRAINT ck_space_name
        CHECK (CHAR_LENGTH(TRIM(space_name)) BETWEEN 1 AND 30),
    CONSTRAINT ck_space_type
        CHECK (space_type IN (1, 2, 3, 4))
) ENGINE = InnoDB;
```

設計重點：

- `space_id` 使用數字代理鍵，作為其他資料表的外鍵參照目標。
- `space_code` 使用 ASCII 與二進位定序，確保固定代碼大小寫與格式一致。
- `space_type` 以 `TINYINT UNSIGNED` 儲存固定分類，並以 `CHECK` 限制有效代碼。

6.3 維護廠商 (`vendor`)

Schema 對照表

資料型態	欄位名稱	鍵值 / 限制	欄位說明
<code>SMALLINT UNSIGNED AUTO_INCREMENT</code>	<code>vendor_id</code>	<code>NOT NULL, AUTO_INCREMENT</code>	維護廠商編號。
<code>VARCHAR(60)</code>	<code>vendor_name</code>	<code>NOT NULL</code>	廠商名稱。
<code>VARCHAR(12) CHARACTER SET ascii COLLATE ascii_bin</code>	<code>contact_phone</code>	<code>NOT NULL</code>	廠商聯繫方式。
<code>TINYINT UNSIGNED</code>	<code>is_active</code>	<code>NOT NULL, DEFAULT 1</code>	啟用狀態： <code>0</code> = 停用、 <code>1</code> = 啟用。
表級約束／索引	<code>pk_vendor</code>	<code>PRIMARY KEY</code>	以 <code>vendor_id</code> 作為廠商主鍵。
表級約束／索引	<code>uq_vendor_name</code>	<code>UNIQUE</code>	確保廠商名稱不重複。
表級約束／索引	<code>ck_vendor_name</code>	<code>CHECK: CHAR_LENGTH(TRIM(vendor_name)) BETWEEN 2 AND 60</code>	避免廠商名稱過短、空白或過長。
表級約束／索引	<code>ck_vendor_phone</code>	<code>CHECK: contact_phone REGEXP '^0(2-[0-9]{4} [3-8]-[0-9]{3})-[0-9]{4}\$' OR contact_phone REGEXP '^09[0-9]{2}-[0-9]{3}-[0-9]{3}\$'</code>	限制電話格式，例如 <code>02-2345-1001</code> 、 <code>05-631-1234</code> 、 <code>0912-345-678</code> 。
表級約束／索引	<code>ck_vendor_active</code>	<code>CHECK: is_active IN (0, 1)</code>	限制啟用狀態只能為 <code>0</code> 或 <code>1</code> 。

對應 SQL：CREATE TABLE vendor

```
CREATE TABLE vendor (  
  vendor_id SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,  
  vendor_name VARCHAR(60) NOT NULL,  
  contact_phone VARCHAR(12)  
    CHARACTER SET ascii COLLATE ascii_bin NOT NULL,  
  is_active TINYINT UNSIGNED NOT NULL DEFAULT 1,  
  
  CONSTRAINT pk_vendor PRIMARY KEY (vendor_id),  
  CONSTRAINT uq_vendor_name UNIQUE (vendor_name),  
  
  CONSTRAINT ck_vendor_name  
    CHECK (CHAR_LENGTH(TRIM(vendor_name)) BETWEEN 2 AND 60),  
  
  CONSTRAINT ck_vendor_phone  
    CHECK (  
      contact_phone REGEXP  
        '^0(2-[0-9]{4}|[3-8]-[0-9]{3})-[0-9]{4}$'  
      OR contact_phone REGEXP '^09[0-9]{2}-[0-9]{3}-[0-9]{3}$'  
    ),  
  
  CONSTRAINT ck_vendor_active  
    CHECK (is_active IN (0, 1))  
) ENGINE = InnoDB;
```

設計重點：

- 廠商主鍵使用 SMALLINT UNSIGNED AUTO_INCREMENT，可涵蓋系所規模的廠商資料。
- contact_phone 使用 ASCII 欄位並以 CHECK 驗證電話格式。
- is_active 使用 DEFAULT 1，新建廠商預設為可使用狀態。

6.4 使用者 (app_user)

Schema 對照表

資料型態	欄位名稱	鍵值 / 限制	欄位說明
MEDIUMINT UNSIGNED AUTO_INCREMENT	user_id	NOT NULL, AUTO_INCREMENT	系統使用者流水主鍵。
CHAR(5) CHARACTER SET ascii COLLATE ascii_bin	user_code	NOT NULL	使用者編號。
VARCHAR(40)	user_name	NOT NULL	使用者姓名。
VARCHAR(80) CHARACTER SET ascii COLLATE ascii_general_ci	email	NOT NULL	聯絡方式。
TINYINT UNSIGNED	role_id	NOT NULL	角色代碼，參照 role_type；1 = 系所管理員、 2 = 全系師生、3 = 設備負責人。
TINYINT UNSIGNED	is_active	NOT NULL, DEFAULT 1	帳號啟用狀態：0 = 停用、1 = 啟用。
DATETIME	created_at	NOT NULL, DEFAULT CURRENT_TIMESTAMP	使用者建立時間。

資料型態	欄位名稱	鍵值 / 限制	欄位說明
表級約束／索引	pk_app_user	PRIMARY KEY	以 user_id 作為使用者主鍵。
表級約束／索引	uq_app_user_code	UNIQUE	確保 user_code 不重複。
表級約束／索引	uq_app_user_email	UNIQUE	確保 email 不重複。
表級約束／索引	ck_app_user_code	CHECK: user_code REGEXP '^U[0-9]{4}\$'	確保使用者代碼符合 U0001 這類固定格式。
表級約束／索引	ck_app_user_name	CHECK: CHAR_LENGTH(TRIM(user_name)) BETWEEN 1 AND 40	避免使用者姓名為空白或過長。
表級約束／索引	ck_app_user_email	CHECK: email REGEXP '^[A-Za-z0-9][A-Za-z0-9._%+-]*@nfu[.]edu[.]tw\$'	限制電子郵件必須為校內帳號。
表級約束／索引	ck_app_user_active	CHECK: is_active IN (0, 1)	限制帳號狀態只能為 0 或 1。
表級約束／索引	fk_app_user_role	FOREIGN KEY, ON DELETE RESTRICT, ON UPDATE RESTRICT	role_id 參照 role_type(role_id) ，確保使用者角色有效。

對應 SQL : CREATE TABLE app_user

```

CREATE TABLE app_user (
    user_id MEDIUMINT UNSIGNED NOT NULL AUTO_INCREMENT,

    user_code CHAR(5) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,

    user_name VARCHAR(40) NOT NULL,

    email VARCHAR(80)
        CHARACTER SET ascii COLLATE ascii_general_ci NOT NULL,

    role_id TINYINT UNSIGNED NOT NULL,
    is_active TINYINT UNSIGNED NOT NULL DEFAULT 1,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT pk_app_user PRIMARY KEY (user_id),
    CONSTRAINT uq_app_user_code UNIQUE (user_code),
    CONSTRAINT uq_app_user_email UNIQUE (email),

    CONSTRAINT ck_app_user_code
        CHECK (user_code REGEXP '^U[0-9]{4}$'),
    CONSTRAINT ck_app_user_name
        CHECK (CHAR_LENGTH(TRIM(user_name)) BETWEEN 1 AND 40),
    CONSTRAINT ck_app_user_email
        CHECK (
            email REGEXP
            '^[A-Za-z0-9][A-Za-z0-9._%+-]*@nfu[.]edu[.]tw$'
        ),
    CONSTRAINT ck_app_user_active
        CHECK (is_active IN (0, 1)),

    CONSTRAINT fk_app_user_role
        FOREIGN KEY (role_id)
        REFERENCES role_type(role_id)
        ON DELETE RESTRICT
        ON UPDATE RESTRICT
) ENGINE = InnoDB;

```

設計重點：

- 使用 `app_user` 作為表名，避免與資料庫保留字 `USER` 衝突。
- `user_code` 作為業務識別碼，`user_id` 作為主鍵與外鍵參照來源。
- `role_id` 透過外鍵連到 `role_type`，並使用 `RESTRICT` 保持角色參照完整性。

6.5 物品 (`item`)

Schema 對照表

資料型態	欄位名稱	鍵值 / 限制	欄位說明
MEDIUMINT UNSIGNED AUTO_INCREMENT	item_id	NOT NULL, AUTO_INCREMENT	物品流水主鍵。
CHAR(5) CHARACTER SET ascii COLLATE ascii_bin	item_code	NOT NULL	內部唯一編號。
VARCHAR(80)	item_name	NOT NULL	名稱。
TINYINT UNSIGNED	item_type	NOT NULL	物品類型代碼：1 = 財產設備、2 = 非列管設備、3 = 耗材。
TINYINT UNSIGNED	current_status	NOT NULL, DEFAULT 1	狀態。
TINYINT UNSIGNED	is_borrowable	NOT NULL, DEFAULT 0	財產設備是否可借用：0 = 不可借、1 = 可借；非列管設備借用判斷以 reusable_equipment.is_borrowable 為準，耗材必須固定為 0。
DATE	warranty_expiry	NULL	保固期限。
SMALLINT UNSIGNED	space_id	NOT NULL	物品所在空間，參照 space(space_id)。
MEDIUMINT UNSIGNED	created_by_user_id	NOT NULL	建立此物品資料的使用者，參照 app_user(user_id)。
DATETIME	created_at	NOT NULL, DEFAULT CURRENT_TIMESTAMP	物品資料建立時間。
表級約束／索引	pk_item	PRIMARY KEY	以 item_id 作為物品主鍵。
表級約束／索引	uq_item_code	UNIQUE	確保 item_code 不重複。
表級約束／索引	ck_item_name	CHECK: CHAR_LENGTH(TRIM(item_name)) BETWEEN 1 AND 80	避免物品名稱為空白或過長。
表級約束／索引	ck_item_type	CHECK: item_type IN (1, 2, 3)	限制物品類型只能使用定義好的三種代碼。
表級約束／索引	ck_item_status	CHECK: current_status IN (1, 2, 3, 4, 5, 6)	限制物品狀態只能使用定義好的六種代碼。
表級約束／索引	ck_item_borrowable	CHECK: is_borrowable IN (0, 1)	限制是否可借用只能為 0 或 1。
表級約束／索引	ck_consumable_not_borrowable	CHECK: item_type <> 3 OR is_borrowable = 0	確保耗材不得進入借用流程。
表級約束／索引	ck_item_code_type	CHECK: (item_type = 1 AND item_code REGEXP '^A[0-9]{4}\$') OR (item_type = 2 AND item_code REGEXP '^R[0-9]{4}\$') OR (item_type = 3 AND item_code REGEXP '^C[0-9]{4}\$')	財產設備使用 A0001 格式，非列管設備使用 R0001 格式，耗材使用 C0001 格式。
表級約束／索引	fk_item_space	FOREIGN KEY, ON DELETE RESTRICT, ON UPDATE RESTRICT	space_id 參照 space(space_id)，確保物品存放空間有效。
表級約束／索引	fk_item_created_by	FOREIGN KEY, ON DELETE RESTRICT, ON UPDATE RESTRICT	created_by_user_id 參照 app_user(user_id)，保留建立者可追溯性。
表級約束／索引	idx_item_space_status	INDEX	建立 (space_id, current_status) 索引，加速依空間與狀態查詢物品。
表級約束／索引	idx_item_type_status	INDEX	建立 (item_type, current_status) 索引，加速依物品類型與狀態查詢。

對應 SQL：CREATE TABLE item

```

CREATE TABLE item (
    item_id MEDIUMINT UNSIGNED NOT NULL AUTO_INCREMENT,

    item_code CHAR(5) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,

    item_name VARCHAR(80) NOT NULL,
    item_type TINYINT UNSIGNED NOT NULL,
    current_status TINYINT UNSIGNED NOT NULL DEFAULT 1,

    is_borrowable TINYINT UNSIGNED NOT NULL DEFAULT 0,

    warranty_expiry DATE NULL,

    space_id SMALLINT UNSIGNED NOT NULL,
    created_by_user_id MEDIUMINT UNSIGNED NOT NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT pk_item PRIMARY KEY (item_id),
    CONSTRAINT uq_item_code UNIQUE (item_code),

    CONSTRAINT ck_item_name
        CHECK (CHAR_LENGTH(TRIM(item_name)) BETWEEN 1 AND 80),

    CONSTRAINT ck_item_type
        CHECK (item_type IN (1, 2, 3)),

    CONSTRAINT ck_item_status
        CHECK (current_status IN (1, 2, 3, 4, 5, 6)),

    CONSTRAINT ck_item_borrowable
        CHECK (is_borrowable IN (0, 1)),

    CONSTRAINT ck_consumable_not_borrowable
        CHECK (item_type <> 3 OR is_borrowable = 0),

    CONSTRAINT ck_item_code_type
        CHECK (
            (item_type = 1 AND item_code REGEXP '^A[0-9]{4}$')
            OR
            (item_type = 2 AND item_code REGEXP '^R[0-9]{4}$')
            OR
            (item_type = 3 AND item_code REGEXP '^C[0-9]{4}$')
        ),

    CONSTRAINT fk_item_space
        FOREIGN KEY (space_id)
        REFERENCES space(space_id)
        ON DELETE RESTRICT

```



```

ON UPDATE RESTRICT,

CONSTRAINT fk_item_created_by
    FOREIGN KEY (created_by_user_id)
    REFERENCES app_user(user_id)
    ON DELETE RESTRICT
    ON UPDATE RESTRICT,

INDEX idx_item_space_status (space_id, current_status),
INDEX idx_item_type_status (item_type, current_status)
) ENGINE = InnoDB;

```

設計重點：

- `item_id` 作為主鍵，`item_code` 作為唯一的業務識別碼。
- 借用判斷規則：財產設備以 `item.is_borrowable` 為準，非列管設備以 `reusable_equipment.is_borrowable` 為準，耗材不得借用。
- 針對常見查詢路徑建立 `(space_id, current_status)` 與 `(item_type, current_status)` 索引。

6.6 財產設備 (`asset_detail`)

Schema 對照表

資料型態	欄位名稱	鍵值 / 限制	欄位說明
MEDIUMINT UNSIGNED	<code>item_id</code>	NOT NULL	對應物品主檔的 <code>item_id</code> ，同時作為本表主鍵。
CHAR(8) CHARACTER SET ascii COLLATE ascii_bin	<code>asset_code</code>	NOT NULL	財產編號。
TINYINT UNSIGNED	<code>fund_source_code</code>	NOT NULL	經費來源。
VARCHAR(60)	<code>fund_source_note</code>	NULL	當 <code>fund_source_code = 5</code> 時必填，其他經費來源則必須為空。
DATE	<code>acquired_date</code>	NOT NULL	取得日期。
INT UNSIGNED	<code>acquired_cost</code>	NOT NULL	取得金額。
TINYINT UNSIGNED	<code>useful_life_years</code>	NOT NULL	耐用年限。
MEDIUMINT UNSIGNED	<code>custodian_user_id</code>	NOT NULL	保管人。
表級約束／索引	<code>pk_asset_detail</code>	PRIMARY KEY	以 <code>item_id</code> 作為財產設備子表主鍵。
表級約束／索引	<code>uq_asset_code</code>	UNIQUE	確保 <code>asset_code</code> 不重複。
表級約束／索引	<code>ck_asset_code</code>	CHECK: <code>asset_code REGEXP '^P[0-9]{7}\$'</code>	確保財產代碼符合 P2024001 這類固定格式。
表級約束／索引	<code>ck_asset_fund_source</code>	CHECK: <code>fund_source_code IN (1, 2, 3, 4, 5)</code>	限制經費來源只能使用定義好的五種代碼。

資料型態	欄位名稱	鍵值 / 限制	欄位說明
表級約束／索引	ck_asset_fund_source_note	CHECK: (fund_source_code = 5 AND fund_source_note IS NOT NULL) OR (fund_source_code <> 5 AND fund_source_note IS NULL)	確保「其他」經費來源需補充說明，非「其他」不得填寫說明。
表級約束／索引	ck_asset_cost	CHECK: acquired_cost >= 3000	財產設備取得金額需達列管門檻。
表級約束／索引	ck_asset_life	CHECK: useful_life_years BETWEEN 1 AND 50	限制耐用年限在合理範圍內。
表級約束／索引	fk_asset_item	FOREIGN KEY, ON DELETE RESTRICT, ON UPDATE RESTRICT	item_id 參照 item(item_id)，確保財產設備必須存在於物品主檔。
表級約束／索引	fk_asset_custodian	FOREIGN KEY, ON DELETE RESTRICT, ON UPDATE RESTRICT	custodian_user_id 參照 app_user(user_id)，確保保管人為有效使用者。

對應 SQL：CREATE TABLE asset_detail

```

CREATE TABLE asset_detail (
    item_id MEDIUMINT UNSIGNED NOT NULL,

    asset_code CHAR(8) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,

    fund_source_code TINYINT UNSIGNED NOT NULL,
    fund_source_note VARCHAR(60) NULL,

    acquired_date DATE NOT NULL,

    acquired_cost INT UNSIGNED NOT NULL,

    useful_life_years TINYINT UNSIGNED NOT NULL,
    custodian_user_id MEDIUMINT UNSIGNED NOT NULL,

    CONSTRAINT pk_asset_detail PRIMARY KEY (item_id),
    CONSTRAINT uq_asset_code UNIQUE (asset_code),

    CONSTRAINT ck_asset_code
        CHECK (asset_code REGEXP '^P[0-9]{7}$'),

    CONSTRAINT ck_asset_fund_source
        CHECK (fund_source_code IN (1, 2, 3, 4, 5)),

    CONSTRAINT ck_asset_fund_source_note
        CHECK (
            (fund_source_code = 5 AND fund_source_note IS NOT NULL)
            OR
            (fund_source_code <> 5 AND fund_source_note IS NULL)
        ),

    CONSTRAINT ck_asset_cost
        CHECK (acquired_cost >= 3000),

    CONSTRAINT ck_asset_life
        CHECK (useful_life_years BETWEEN 1 AND 50),

    CONSTRAINT fk_asset_item
        FOREIGN KEY (item_id)
        REFERENCES item(item_id)
        ON DELETE RESTRICT
        ON UPDATE RESTRICT,

    CONSTRAINT fk_asset_custodian
        FOREIGN KEY (custodian_user_id)
        REFERENCES app_user(user_id)
        ON DELETE RESTRICT

```

```
ON UPDATE RESTRICT
```

```
) ENGINE = InnoDB;
```

設計重點：

- `item_id` 採父表主鍵作為子表主鍵，形成 `item` 與 `asset_detail` 的一對一分類資料。
- `asset_code` 使用固定格式代碼並加上 `UNIQUE`，符合財產編號不可重複的需求。
- 經費來源使用代碼加上條件式 `CHECK`，確保只有「其他」才需要填寫補充說明。

6.7 可重複使用的非列管設備 (`reusable_equipment`)

Schema 對照表

資料型態	欄位名稱	鍵值 / 限制	欄位說明
MEDIUMINT UNSIGNED	<code>item_id</code>	NOT NULL	對應物品主檔的 <code>item_id</code> ，同時作為本表主鍵。
VARCHAR(120)	<code>specification</code>	NOT NULL	規格。
SMALLINT UNSIGNED	<code>quantity</code>	NOT NULL, DEFAULT 1	數量。
TINYINT UNSIGNED	<code>is_borrowable</code>	NOT NULL, DEFAULT 1	可借用；非列管設備借用判斷以此欄為準。
TINYINT UNSIGNED	<code>need_return</code>	NOT NULL, DEFAULT 1	需歸還。
表級約束／索引	<code>pk_reusable_equipment</code>	PRIMARY KEY	以 <code>item_id</code> 作為非列管設備子表主鍵。
表級約束／索引	<code>ck_reusable_specification</code>	CHECK: CHAR_LENGTH(TRIM(specification)) BETWEEN 1 AND 120	避免規格內容為空白或過長。
表級約束／索引	<code>ck_reusable_is_borrowable</code>	CHECK: is_borrowable IN (0, 1)	限制可借用欄位只能為 0 或 1。
表級約束／索引	<code>ck_reusable_need_return</code>	CHECK: need_return IN (0, 1)	限制需歸還欄位只能為 0 或 1。
表級約束／索引	<code>fk_reusable_item</code>	FOREIGN KEY, ON DELETE RESTRICT, ON UPDATE RESTRICT	<code>item_id</code> 參照 <code>item(item_id)</code> ，確保非列管設備必須存在於物品主檔。

對應 SQL：CREATE TABLE `reusable_equipment`

```

CREATE TABLE reusable_equipment (
    item_id MEDIUMINT UNSIGNED NOT NULL,
    specification VARCHAR(120) NOT NULL,
    quantity SMALLINT UNSIGNED NOT NULL DEFAULT 1,
    is_borrowable TINYINT UNSIGNED NOT NULL DEFAULT 1,
    need_return TINYINT UNSIGNED NOT NULL DEFAULT 1,

    CONSTRAINT pk_reusable_equipment PRIMARY KEY (item_id),

    CONSTRAINT ck_reusable_specification
        CHECK (CHAR_LENGTH(TRIM(specification)) BETWEEN 1 AND 120),

    CONSTRAINT ck_reusable_is_borrowable
        CHECK (is_borrowable IN (0, 1)),

    CONSTRAINT ck_reusable_need_return
        CHECK (need_return IN (0, 1)),

    CONSTRAINT fk_reusable_item
        FOREIGN KEY (item_id)
        REFERENCES item(item_id)
        ON DELETE RESTRICT
        ON UPDATE RESTRICT
) ENGINE = InnoDB;

```

設計重點：

- 子表保存非列管設備的 `specification`、`quantity`、`is_borrowable` 與 `need_return`。
- `item_id` 同時作為主鍵與外鍵，確保每筆非列管設備只對應一筆物品主檔。
- 外鍵採 `ON DELETE RESTRICT`，保留已有子表或歷程資料的物品紀錄。

6.8 耗材 (`consumable_detail`)

Schema 對照表

資料型態	欄位名稱	鍵值 / 限制	欄位說明
MEDIUMINT UNSIGNED	<code>item_id</code>	NOT NULL	對應物品主檔的 <code>item_id</code> ，同時作為本表主鍵。
MEDIUMINT UNSIGNED	<code>stock_quantity</code>	NOT NULL, DEFAULT 0	庫存量。
MEDIUMINT UNSIGNED	<code>min_stock</code>	NOT NULL, DEFAULT 0	最低庫存量。
TINYINT UNSIGNED	<code>unit_code</code>	NOT NULL	單位。
表級約束／索引	<code>pk_consumable_detail</code>	PRIMARY KEY	以 <code>item_id</code> 作為耗材子表主鍵。
表級約束／索引	<code>ck_consumable_unit</code>	CHECK: unit_code IN (1, 2, 3, 4, 5, 6, 7, 8)	限制耗材單位只能使用定義好的八種代碼。

資料型態	欄位名稱	鍵值 / 限制	欄位說明
表級約束／索引	fk_consumable_item	FOREIGN KEY, ON DELETE RESTRICT, ON UPDATE RESTRICT	item_id 參照 item(item_id) ，確保耗材必須存在於物品主檔。

對應 SQL：`CREATE TABLE consumable_detail`

```
CREATE TABLE consumable_detail (  
    item_id MEDIUMINT UNSIGNED NOT NULL,  
  
    stock_quantity MEDIUMINT UNSIGNED NOT NULL DEFAULT 0,  
    min_stock MEDIUMINT UNSIGNED NOT NULL DEFAULT 0,  
    unit_code TINYINT UNSIGNED NOT NULL,  
  
    CONSTRAINT pk_consumable_detail PRIMARY KEY (item_id),  
  
    CONSTRAINT ck_consumable_unit  
        CHECK (unit_code IN (1, 2, 3, 4, 5, 6, 7, 8)),  
  
    CONSTRAINT fk_consumable_item  
        FOREIGN KEY (item_id)  
        REFERENCES item(item_id)  
        ON DELETE RESTRICT  
        ON UPDATE RESTRICT  
)  
ENGINE = InnoDB;
```

設計重點：

- stock_quantity 與 min_stock 使用 MEDIUMINT UNSIGNED ，兼顧足夠容量與儲存效率。
- unit_code 使用固定代碼，維持耗材單位資料一致。
- 以 item_id 連回 item 主檔，維持耗材分類資料與共用物品資料一致。

6.9 領用紀錄 (consume_record)

Schema 對照表

資料型態	欄位名稱	鍵值 / 限制	欄位說明
INT UNSIGNED AUTO_INCREMENT	consume_id	NOT NULL, AUTO_INCREMENT	領用紀錄單號。
MEDIUMINT UNSIGNED	item_id	NOT NULL	被領用的耗材，參照 consumable_detail(item_id) 。
MEDIUMINT UNSIGNED	user_id	NOT NULL	領用者，參照 app_user(user_id) 。
DATETIME	consume_time	NOT NULL, DEFAULT CURRENT_TIMESTAMP	領用時間。
MEDIUMINT UNSIGNED	amount	NOT NULL	數量。
VARCHAR(80)	purpose	NOT NULL	用途。
表級約束／索引	pk_consume_record	PRIMARY KEY	以 consume_id 作為耗材領用紀錄主鍵。
表級約束／索引	ck_consume_amount	CHECK: amount >= 1	確保領用數量為正整數。

資料型態	欄位名稱	鍵值 / 限制	欄位說明
表級約束／索引	ck_consume_purpose	CHECK: CHAR_LENGTH(TRIM(purpose)) BETWEEN 2 AND 80	避免用途說明為空白、過短或過長。
表級約束／索引	fk_consume_item	FOREIGN KEY, ON DELETE RESTRICT, ON UPDATE RESTRICT	item_id 參照 consumable_detail(item_id) · 確保只能領用有效耗材。
表級約束／索引	fk_consume_user	FOREIGN KEY, ON DELETE RESTRICT, ON UPDATE RESTRICT	user_id 參照 app_user(user_id) · 保留領用者可追溯性。
表級約束／索引	idx_consume_item_time	INDEX	建立 (item_id, consume_time) 索引 · 加速查詢單一耗材的領用歷程。
表級約束／索引	idx_consume_user_time	INDEX	建立 (user_id, consume_time) 索引 · 加速查詢使用者領用紀錄。

對應 SQL：CREATE TABLE consume_record

```
CREATE TABLE consume_record (  
  consume_id INT UNSIGNED NOT NULL AUTO_INCREMENT,  
  item_id MEDIUMINT UNSIGNED NOT NULL,  
  user_id MEDIUMINT UNSIGNED NOT NULL,  
  
  consume_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  
  amount MEDIUMINT UNSIGNED NOT NULL,  
  
  purpose VARCHAR(80) NOT NULL,  
  
  CONSTRAINT pk_consume_record PRIMARY KEY (consume_id),  
  
  CONSTRAINT ck_consume_amount  
    CHECK (amount >= 1),  
  
  CONSTRAINT ck_consume_purpose  
    CHECK (CHAR_LENGTH(TRIM(purpose)) BETWEEN 2 AND 80),  
  
  CONSTRAINT fk_consume_item  
    FOREIGN KEY (item_id)  
    REFERENCES consumable_detail(item_id)  
    ON DELETE RESTRICT  
    ON UPDATE RESTRICT,  
  
  CONSTRAINT fk_consume_user  
    FOREIGN KEY (user_id)  
    REFERENCES app_user(user_id)  
    ON DELETE RESTRICT  
    ON UPDATE RESTRICT,  
  
  INDEX idx_consume_item_time (item_id, consume_time),  
  INDEX idx_consume_user_time (user_id, consume_time)  
) ENGINE = InnoDB;
```

設計重點：

- `item_id` 指向 `consumable_detail`，從資料庫層限制領用紀錄只能對應耗材。
- `amount` 使用 `MEDIUMINT UNSIGNED` 並加上 `CHECK`，限制領用數量必須大於 0。
- 依耗材與使用者兩種查詢路徑建立時間索引，支援庫存稽核與個人領用追蹤。

6.10 借用紀錄 (`borrow_record`)

Schema 對照表

資料型態	欄位名稱	鍵值 / 限制	欄位說明
INT UNSIGNED AUTO_INCREMENT	<code>borrow_id</code>	NOT NULL, AUTO_INCREMENT	借用紀錄單號。
MEDIUMINT UNSIGNED	<code>item_id</code>	NOT NULL	被借用的物品，參照 <code>item(item_id)</code> 。
MEDIUMINT UNSIGNED	<code>user_id</code>	NOT NULL	借用者，參照 <code>app_user(user_id)</code> 。
DATETIME	<code>borrow_time</code>	NOT NULL, DEFAULT CURRENT_TIMESTAMP	借用時間。
DATETIME	<code>expected_return_time</code>	NOT NULL	預計歸還時間。
DATETIME	<code>actual_return_time</code>	NULL	實際歸還時間。
TINYINT UNSIGNED	<code>borrow_status</code>	NOT NULL, DEFAULT 1	借用狀態。
表級約束／索引	<code>pk_borrow_record</code>	PRIMARY KEY	以 <code>borrow_id</code> 作為借用紀錄主鍵。
表級約束／索引	<code>ck_borrow_status</code>	CHECK: <code>borrow_status</code> IN (1, 2, 3)	限制借用狀態只能使用定義好的三種代碼。
表級約束／索引	<code>ck_borrow_expected_time</code>	CHECK: <code>expected_return_time</code> > <code>borrow_time</code>	確保預計歸還時間晚於借用時間。
表級約束／索引	<code>ck_borrow_actual_time</code>	CHECK: <code>actual_return_time</code> IS NULL OR <code>actual_return_time</code> >= <code>borrow_time</code>	確保實際歸還時間不早於借用時間。
表級約束／索引	<code>ck_borrow_status_return</code>	CHECK: (<code>borrow_status</code> IN (1, 3) AND <code>actual_return_time</code> IS NULL) OR (<code>borrow_status</code> = 2 AND <code>actual_return_time</code> IS NOT NULL)	借用中或逾期時不得有實際歸還時間，已歸還時必須有實際歸還時間。
表級約束／索引	<code>fk_borrow_item</code>	FOREIGN KEY, ON DELETE RESTRICT, ON UPDATE RESTRICT	<code>item_id</code> 參照 <code>item(item_id)</code> ，保留設備借用歷程。
表級約束／索引	<code>fk_borrow_user</code>	FOREIGN KEY, ON DELETE RESTRICT, ON UPDATE RESTRICT	<code>user_id</code> 參照 <code>app_user(user_id)</code> ，保留借用者可追溯性。
表級約束／索引	<code>idx_borrow_item_open</code>	INDEX	建立 (<code>item_id</code> , <code>actual_return_time</code>) 索引，加速查詢設備未歸還紀錄。
表級約束／索引	<code>idx_borrow_user_time</code>	INDEX	建立 (<code>user_id</code> , <code>borrow_time</code>) 索引，加速查詢使用者借用歷程。

對應 SQL：CREATE TABLE `borrow_record`


```

CREATE TABLE borrow_record (
    borrow_id INT UNSIGNED NOT NULL AUTO_INCREMENT,
    item_id MEDIUMINT UNSIGNED NOT NULL,
    user_id MEDIUMINT UNSIGNED NOT NULL,

    borrow_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    expected_return_time DATETIME NOT NULL,
    actual_return_time DATETIME NULL,

    borrow_status TINYINT UNSIGNED NOT NULL DEFAULT 1,

    CONSTRAINT pk_borrow_record PRIMARY KEY (borrow_id),

    CONSTRAINT ck_borrow_status
        CHECK (borrow_status IN (1, 2, 3)),

    CONSTRAINT ck_borrow_expected_time
        CHECK (expected_return_time > borrow_time),

    CONSTRAINT ck_borrow_actual_time
        CHECK (
            actual_return_time IS NULL
            OR actual_return_time >= borrow_time
        ),

    CONSTRAINT ck_borrow_status_return
        CHECK (
            (borrow_status IN (1, 3) AND actual_return_time IS NULL)
            OR
            (borrow_status = 2 AND actual_return_time IS NOT NULL)
        ),

    CONSTRAINT fk_borrow_item
        FOREIGN KEY (item_id)
        REFERENCES item(item_id)
        ON DELETE RESTRICT
        ON UPDATE RESTRICT,

    CONSTRAINT fk_borrow_user
        FOREIGN KEY (user_id)
        REFERENCES app_user(user_id)
        ON DELETE RESTRICT
        ON UPDATE RESTRICT,

    INDEX idx_borrow_item_open (item_id, actual_return_time),
    INDEX idx_borrow_user_time (user_id, borrow_time)
) ENGINE = InnoDB;

```

設計重點：

- `borrow_status` 保留實際借出後會發生的狀態。
- 時間欄位透過 `CHECK` 維持借用、預計歸還與實際歸還的先後邏輯。
- `(item_id, actual_return_time)` 索引可快速找出特定設備目前是否仍未歸還。

6.11 維修工單 (`maintenance_ticket`)

Schema 對照表

資料型態	欄位名稱	鍵值 / 限制	欄位說明
INT UNSIGNED AUTO_INCREMENT	<code>ticket_id</code>	NOT NULL, AUTO_INCREMENT	維修工單編號。
MEDIUMINT UNSIGNED	<code>item_id</code>	NOT NULL	維修對象，參照 <code>item(item_id)</code> 。
MEDIUMINT UNSIGNED	<code>reporter_user_id</code>	NOT NULL	報修者，參照 <code>app_user(user_id)</code> 。
MEDIUMINT UNSIGNED	<code>handler_user_id</code>	NULL	處理者，參照 <code>app_user(user_id)</code> ，尚未指派時可為空。
DATETIME	<code>reported_time</code>	NOT NULL, DEFAULT CURRENT_TIMESTAMP	報修日期。
TEXT	<code>issue_desc</code>	NOT NULL	故障描述。
TINYINT UNSIGNED	<code>maintenance_status</code>	NOT NULL, DEFAULT 1	工單狀態。
表級約束／索引	<code>pk_maintenance_ticket</code>	PRIMARY KEY	以 <code>ticket_id</code> 作為維修工單主鍵。
表級約束／索引	<code>ck_ticket_issue_desc</code>	CHECK: CHAR_LENGTH(TRIM(issue_desc)) BETWEEN 5 AND 2000	限制故障描述需有足夠內容且不可過長。
表級約束／索引	<code>ck_ticket_status</code>	CHECK: maintenance_status IN (1, 2, 3, 4)	限制維修狀態只能使用定義好的四種代碼。
表級約束／索引	<code>fk_ticket_item</code>	FOREIGN KEY, ON DELETE RESTRICT, ON UPDATE RESTRICT	<code>item_id</code> 參照 <code>item(item_id)</code> ，保留維修對象可追溯性。
表級約束／索引	<code>fk_ticket_reporter</code>	FOREIGN KEY, ON DELETE RESTRICT, ON UPDATE RESTRICT	<code>reporter_user_id</code> 參照 <code>app_user(user_id)</code> ，保留報修者紀錄。
表級約束／索引	<code>fk_ticket_handler</code>	FOREIGN KEY, ON DELETE RESTRICT, ON UPDATE RESTRICT	<code>handler_user_id</code> 參照 <code>app_user(user_id)</code> ，保留處理者紀錄。
表級約束／索引	<code>idx_ticket_item_status</code>	INDEX	建立 <code>(item_id, maintenance_status)</code> 索引，加速查詢物品維修狀態。
表級約束／索引	<code>idx_ticket_handler_status</code>	INDEX	建立 <code>(handler_user_id, maintenance_status)</code> 索引，加速查詢處理者待辦工單。

對應 SQL：`CREATE TABLE maintenance_ticket`

```

CREATE TABLE maintenance_ticket (
    ticket_id INT UNSIGNED NOT NULL AUTO_INCREMENT,
    item_id MEDIUMINT UNSIGNED NOT NULL,

    reporter_user_id MEDIUMINT UNSIGNED NOT NULL,
    handler_user_id MEDIUMINT UNSIGNED NULL,

    reported_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,

    issue_desc TEXT NOT NULL,

    maintenance_status TINYINT UNSIGNED NOT NULL DEFAULT 1,

    CONSTRAINT pk_maintenance_ticket PRIMARY KEY (ticket_id),

    CONSTRAINT ck_ticket_issue_desc
        CHECK (CHAR_LENGTH(TRIM(issue_desc)) BETWEEN 5 AND 2000),

    CONSTRAINT ck_ticket_status
        CHECK (maintenance_status IN (1, 2, 3, 4)),

    CONSTRAINT fk_ticket_item
        FOREIGN KEY (item_id)
        REFERENCES item(item_id)
        ON DELETE RESTRICT
        ON UPDATE RESTRICT,

    CONSTRAINT fk_ticket_reporter
        FOREIGN KEY (reporter_user_id)
        REFERENCES app_user(user_id)
        ON DELETE RESTRICT
        ON UPDATE RESTRICT,

    CONSTRAINT fk_ticket_handler
        FOREIGN KEY (handler_user_id)
        REFERENCES app_user(user_id)
        ON DELETE RESTRICT
        ON UPDATE RESTRICT,

    INDEX idx_ticket_item_status (item_id, maintenance_status),
    INDEX idx_ticket_handler_status (handler_user_id, maintenance_status)
) ENGINE = InnoDB;

```

設計重點：

- `maintenance_status` 使用固定代碼與 `CHECK`，區分待處理、處理中、已完成與取消。
- 維修工單只保留報修、維修對象、報修者與目前處理者；維修處理細節改由 `maintenance_process_record` 記錄。

- 分別針對物品與處理者建立狀態索引，支援設備維修歷程與負責人待辦查詢。

6.12 維修處理紀錄 (maintenance_process_record)

Schema 對照表

資料型態	欄位名稱	鍵值 / 限制	欄位說明
INT UNSIGNED AUTO_INCREMENT	process_id	NOT NULL, AUTO_INCREMENT	維修處理編號。
INT UNSIGNED	ticket_id	NOT NULL	所屬維修工單，參照 maintenance_ticket(ticket_id)。
SMALLINT UNSIGNED	vendor_id	NULL	維護廠商，參照 vendor(vendor_id)；系內自行處理時可為 NULL。
DATETIME	process_time	NOT NULL, DEFAULT CURRENT_TIMESTAMP	維修處理開始或登錄時間。
INT UNSIGNED	repair_cost	NULL	維修費用。
DATETIME	completed_time	NULL	完成日期。
VARCHAR(120)	replaced_parts	NULL	更換零件。
DATE	next_maintenance_date	NULL	下次保養日期。
VARCHAR(500)	repair_result	NULL	維修結果。
表級約束／索引	pk_maintenance_process_record	PRIMARY KEY	以 process_id 作為維修處理紀錄主鍵。
表級約束／索引	ck_process_cost	CHECK: repair_cost IS NULL OR repair_cost >= 0	限制維修費用不可為負值。
表級約束／索引	ck_process_completed_time	CHECK: completed_time IS NULL OR completed_time >= process_time	確保完成時間不得早於處理時間。
表級約束／索引	ck_process_after_reported_time	CHECK (由 Trigger 或 Procedure 實作)	process_time 不得早於所屬 maintenance_ticket.reported_time；此為跨表時序限制，MySQL/MariaDB 的 CHECK 無法直接查詢另一張表。
表級約束／索引	ck_process_next_maintenance_date	CHECK: next_maintenance_date IS NULL OR (completed_time IS NOT NULL AND next_maintenance_date > DATE(completed_time))	確保下次保養日期必須晚於完成時間。
表級約束／索引	fk_process_ticket	FOREIGN KEY, ON DELETE RESTRICT, ON UPDATE RESTRICT	ticket_id 參照 maintenance_ticket(ticket_id)，維持處理紀錄與工單的參照完整性。
表級約束／索引	fk_process_vendor	FOREIGN KEY, ON DELETE RESTRICT, ON UPDATE RESTRICT	vendor_id 參照 vendor(vendor_id)，並允許系內處理時為 NULL。
表級約束／索引	idx_process_ticket_time	INDEX	建立 (ticket_id, process_time) 索引，加速查詢單一工單的處理歷程。
表級約束／索引	idx_process_vendor_time	INDEX	建立 (vendor_id, process_time) 索引，加速查詢廠商處理紀錄。

對應 SQL：CREATE TABLE maintenance_process_record

```

CREATE TABLE maintenance_process_record (
    process_id INT UNSIGNED NOT NULL AUTO_INCREMENT,
    ticket_id INT UNSIGNED NOT NULL,
    vendor_id SMALLINT UNSIGNED NULL,

    process_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    repair_cost INT UNSIGNED NULL,
    completed_time DATETIME NULL,
    replaced_parts VARCHAR(120) NULL,
    next_maintenance_date DATE NULL,
    repair_result VARCHAR(500) NULL,

    CONSTRAINT pk_maintenance_process_record PRIMARY KEY (process_id),

    CONSTRAINT ck_process_cost
        CHECK (repair_cost IS NULL OR repair_cost >= 0),

    CONSTRAINT ck_process_completed_time
        CHECK (
            completed_time IS NULL
            OR completed_time >= process_time
        ),

    CONSTRAINT ck_process_next_maintenance
        CHECK (
            next_maintenance_date IS NULL
            OR (
                completed_time IS NOT NULL
                AND next_maintenance_date > DATE(completed_time)
            )
        ),

    CONSTRAINT fk_process_ticket
        FOREIGN KEY (ticket_id)
        REFERENCES maintenance_ticket(ticket_id)
        ON DELETE RESTRICT
        ON UPDATE RESTRICT,

    CONSTRAINT fk_process_vendor
        FOREIGN KEY (vendor_id)
        REFERENCES vendor(vendor_id)
        ON DELETE RESTRICT
        ON UPDATE RESTRICT,

    INDEX idx_process_ticket_time (ticket_id, process_time),
    INDEX idx_process_vendor_time (vendor_id, process_time)
) ENGINE = InnoDB;

```

設計重點：

- 一張維修工單可有零到多筆處理紀錄，支援多次檢修、追蹤或追加處理。
- `vendor_id` 可為 `NULL`，表示該次處理由系內自行完成；若委外則必須參照有效廠商。
- 處理時間、完成時間與下次保養日期分開記錄，可完整呈現維修處理歷程。

6.13 狀態異動紀錄 (`status_history`)

Schema 對照表

資料型態	欄位名稱	鍵值 / 限制	欄位說明
INT UNSIGNED AUTO_INCREMENT	<code>history_id</code>	NOT NULL, AUTO_INCREMENT	異動編號。
MEDIUMINT UNSIGNED	<code>item_id</code>	NOT NULL	發生狀態異動的物品，參照 <code>item(item_id)</code> 。
MEDIUMINT UNSIGNED	<code>operator_user_id</code>	NOT NULL	執行狀態異動的使用者，參照 <code>app_user(user_id)</code> 。
DATETIME	<code>changed_time</code>	NOT NULL, DEFAULT CURRENT_TIMESTAMP	異動時間。
TINYINT UNSIGNED	<code>old_status</code>	NOT NULL	舊狀態。
TINYINT UNSIGNED	<code>new_status</code>	NOT NULL	新狀態。
VARCHAR(120)	<code>reason</code>	NOT NULL	異動原因。
表級約束／索引	<code>pk_status_history</code>	PRIMARY KEY	以 <code>history_id</code> 作為狀態異動紀錄主鍵。
表級約束／索引	<code>ck_history_old_status</code>	CHECK: <code>old_status</code> IN (1, 2, 3, 4, 5, 6)	限制異動前狀態只能使用定義好的六種代碼。
表級約束／索引	<code>ck_history_new_status</code>	CHECK: <code>new_status</code> IN (1, 2, 3, 4, 5, 6)	限制異動後狀態只能使用定義好的六種代碼。
表級約束／索引	<code>ck_history_status_changed</code>	CHECK: <code>old_status</code> <> <code>new_status</code>	確保狀態異動確實發生變化。
表級約束／索引	<code>ck_history_reason</code>	CHECK: CHAR_LENGTH(TRIM(<code>reason</code>)) BETWEEN 2 AND 120	避免異動原因為空白、過短或過長。
表級約束／索引	<code>fk_history_item</code>	FOREIGN KEY, ON DELETE RESTRICT, ON UPDATE RESTRICT	<code>item_id</code> 參照 <code>item(item_id)</code> ，保留物品狀態歷程。
表級約束／索引	<code>fk_history_operator</code>	FOREIGN KEY, ON DELETE RESTRICT, ON UPDATE RESTRICT	<code>operator_user_id</code> 參照 <code>app_user(user_id)</code> ，保留操作者可追溯性。
表級約束／索引	<code>idx_history_item_time</code>	INDEX	建立 (<code>item_id</code> , <code>changed_time</code>) 索引，加速查詢單一物品狀態歷程。
表級約束／索引	<code>idx_history_operator_time</code>	INDEX	建立 (<code>operator_user_id</code> , <code>changed_time</code>) 索引，加速查詢操作者異動紀錄。

對應 SQL：CREATE TABLE `status_history`

```

CREATE TABLE status_history (
    history_id INT UNSIGNED NOT NULL AUTO_INCREMENT,
    item_id MEDIUMINT UNSIGNED NOT NULL,
    operator_user_id MEDIUMINT UNSIGNED NOT NULL,

    changed_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,

    old_status TINYINT UNSIGNED NOT NULL,
    new_status TINYINT UNSIGNED NOT NULL,

    reason VARCHAR(120) NOT NULL,

    CONSTRAINT pk_status_history PRIMARY KEY (history_id),

    CONSTRAINT ck_history_old_status
        CHECK (old_status IN (1, 2, 3, 4, 5, 6)),

    CONSTRAINT ck_history_new_status
        CHECK (new_status IN (1, 2, 3, 4, 5, 6)),

    CONSTRAINT ck_history_status_changed
        CHECK (old_status <> new_status),

    CONSTRAINT ck_history_reason
        CHECK (CHAR_LENGTH(TRIM(reason)) BETWEEN 2 AND 120),

    CONSTRAINT fk_history_item
        FOREIGN KEY (item_id)
        REFERENCES item(item_id)
        ON DELETE RESTRICT
        ON UPDATE RESTRICT,

    CONSTRAINT fk_history_operator
        FOREIGN KEY (operator_user_id)
        REFERENCES app_user(user_id)
        ON DELETE RESTRICT
        ON UPDATE RESTRICT,

    INDEX idx_history_item_time (item_id, changed_time),
    INDEX idx_history_operator_time (operator_user_id, changed_time)
) ENGINE = InnoDB;

```

設計重點：

- 狀態異動紀錄以 `history_id` 作為流水主鍵，歷程資料不需使用複合鍵識別。
- `old_status` 與 `new_status` 都以 `TINYINT UNSIGNED` 儲存，並限制只能使用物品狀態代碼。
- 物品與操作者時間索引可支援設備生命週期追蹤與管理者操作稽核。

七、View 設計與使用說明

本節依第六章 Schema 整理 View 設計。View 作為不同系統角色的查詢介面，負責限制可見資料範圍；需要依使用者身分限制的查詢，則由後端以參數化條件補上。

本專題聚焦於設備、耗材、借用、維修與稽核資料庫設計，因此不納入帳號驗證、密碼儲存或登入流程之實作；`app_user` 與 `role_type` 僅用於記錄系統角色及各項交易、歷程資料的相關人員。

7.1 View 設計與系統使用對照

View	對象	用途	篩選條件與資料保護	系統使用位置
<code>vw_Student_Available_Borrowable_Items</code>	全系師生	查詢可借用設備。	只顯示 <code>current_status = 1</code> ，且 <code>item_type</code> 為 1 財產設備或 2 非列管設備的可借用物品；財產設備以 <code>item.is_borrowable</code> 判斷，非列管設備以 <code>reusable_equipment.is_borrowable</code> 判斷；不顯示取得金額、經費來源、保管人等敏感資料。	<code>/student/items</code>
<code>vw_Student_Current_Borrowed_Items</code>	全系師生	查詢目前借用中或逾期未歸還的設備。	只顯示借用中或逾期且尚未歸還的紀錄，例如 <code>borrow_status IN (1, 3)</code> 且 <code>actual_return_time IS NULL</code> ；後端再依使用者身分加上 <code>user_id = current_user_id</code> 條件，只顯示本人借用紀錄。	<code>/student/returns</code>
<code>vw_Student_Available_Consumables</code>	全系師生	查詢可領用耗材。	只顯示 <code>item_type = 3</code> 、 <code>current_status = 1</code> 且 <code>stock_quantity > 0</code> 的耗材；不顯示採購金額、保管人與其他管理欄位。	<code>/student/consumables</code>
<code>vw_Student_Maintenance_Reportable_Items</code>	全系師生	查詢可回報維修的設備。	只顯示財產設備或非列管設備，排除耗材、報廢項目，以及已有待處理或處理中維修工單的物品；不顯示取得金額、經費來源與保管人資訊。	<code>/student/maintenance/report</code>
<code>vw_Student_Maintenance_Handlers</code>	全系師生	報修時查詢可指派的設備負責人。	只顯示角色為設備負責人的使用者，例如 <code>role_id = 3</code> ；僅回傳負責人識別與姓名，避免暴露不必要的使用者資料。	<code>/student/maintenance/report</code> 、 <code>load_default_maintenance_handler_id</code>
<code>vw_Supervisor_Assigned_Maintenance_Tasks</code>	設備負責人	查看待處理、處理中的維修工單。	只顯示 <code>maintenance_status IN (1, 2)</code> 的資料。後端再依使用者身分加上 <code>handler_id = current_user_id</code> 條件，限制只能查看自己被指派的工單。	<code>/supervisor/tasks</code>
<code>vw_Supervisor_Maintenance_History</code>	設備負責人	查看已完成或取消的維修工單。	只顯示 <code>maintenance_status IN (3, 4)</code> 的歷史工單；後端再依使用者身分加上 <code>handler_id = current_user_id</code> 條件，限制只能查看自己被指派的歷史資料。	<code>/supervisor/history</code>
<code>vw_Admin_Asset_Master</code>	系所管理員	全系財產設備盤點。	顯示財產編號、經費來源、取得金額、耐用年限與保管人等管理欄位，僅供系所管理員使用。	<code>/admin/assets</code>
<code>vw_Admin_Consumable_Alert</code>	系所管理員	低庫存耗材預警。	篩選 <code>item_type = 3</code> 、未報廢且 <code>stock_quantity <= min_stock</code> 的耗材。	目前 <code>equipment_web</code> 的 <code>/admin/consumables</code> 以直接查詢呈現完整耗材清單；此 View 可作為低庫存預警查詢使用。
<code>vw_Admin_Maintenance_Ticket_Master</code>	系所管理員	全系維修工單總覽。	顯示所有維修狀態的工單，可由後端依 <code>maintenance_status</code> 篩選；維護廠商、費用、完成時間與維修結果由最新一筆 <code>maintenance_process_record</code> 補充。	<code>/admin/maintenance</code>
<code>vw_Admin_Audit_Trail</code>	系所管理員	整合使用歷程。	以 <code>UNION ALL</code> 整合狀態異動、設備借用、維修報修與維修處理事件，統一輸出事件時間、事件類型、操作者與事件內容，供追蹤設備流向與責任。	<code>/admin/audit</code>

7.2 05_view.sql View 對應完整 SQL

7.2.1 vw_Student_Available_Borrowable_Items

```
CREATE OR REPLACE VIEW vw_Student_Available_Borrowable_Items AS
SELECT
    i.item_id,
    i.item_code,
    i.item_name,
    i.item_type,
    i.current_status,
    s.space_id,
    s.space_code,
    s.space_name,
    s.space_type,
    r.specification,
    r.quantity,
    r.need_return,
    CASE
        WHEN i.item_type = 1 THEN i.is_borrowable
        WHEN i.item_type = 2 THEN r.is_borrowable
        ELSE 0
    END AS is_borrowable
FROM item i
JOIN space s
    ON i.space_id = s.space_id
LEFT JOIN reusable_equipment r
    ON i.item_id = r.item_id
WHERE i.current_status = 1
    AND (
        (i.item_type = 1 AND i.is_borrowable = 1)
        OR
        (i.item_type = 2 AND r.is_borrowable = 1)
    );
```

7.2.2 vw_Student_Available_Consumables

```
CREATE OR REPLACE VIEW vw_Student_Available_Consumables AS
SELECT
    i.item_id,
    i.item_code,
    i.item_name,
    i.item_type,
    i.current_status,
    s.space_id,
    s.space_code,
    s.space_name,
    s.space_type,
    c.stock_quantity,
    c.min_stock,
    c.unit_code
FROM item i
JOIN consumable_detail c
    ON i.item_id = c.item_id
JOIN space s
    ON i.space_id = s.space_id
WHERE i.item_type = 3
    AND i.current_status = 1
    AND c.stock_quantity > 0;
```

7.2.3 vw_Student_Current_Borrowed_Items

```
CREATE OR REPLACE VIEW vw_Student_Current_Borrowed_Items AS
SELECT
    br.borrow_id,
    br.item_id,
    i.item_code,
    i.item_name,
    i.item_type,
    i.current_status,
    s.space_id,
    s.space_code,
    s.space_name,
    s.space_type,
    r.specification,
    r.need_return,
    br.user_id,
    u.user_code,
    u.user_name,
    br.borrow_time,
    br.expected_return_time,
    br.actual_return_time,
    br.borrow_status
FROM borrow_record br
JOIN item i
    ON br.item_id = i.item_id
JOIN space s
    ON i.space_id = s.space_id
JOIN app_user u
    ON br.user_id = u.user_id
LEFT JOIN reusable_equipment r
    ON i.item_id = r.item_id
WHERE br.borrow_status IN (1, 3)
    AND br.actual_return_time IS NULL;
```

7.2.4 vw_Student_Maintenance_Reportable_Items

```
CREATE OR REPLACE VIEW vw_Student_Maintenance_Reportable_Items AS
SELECT
    i.item_id,
    i.item_code,
    i.item_name,
    i.item_type,
    i.current_status,
    i.warranty_expiry,
    s.space_id,
    s.space_code,
    s.space_name,
    s.space_type
FROM item i
JOIN space s
    ON i.space_id = s.space_id
WHERE i.item_type IN (1, 2)
    AND i.current_status <> 5
    AND NOT EXISTS (
        SELECT 1
        FROM maintenance_ticket mt
        WHERE mt.item_id = i.item_id
            AND mt.maintenance_status IN (1, 2)
    );
```

7.2.5 vw_Student_Maintenance_Handlers

```
CREATE OR REPLACE VIEW vw_Student_Maintenance_Handlers AS
SELECT
    u.user_id AS handler_id,
    u.user_code AS handler_code,
    u.user_name AS handler_name
FROM app_user u
JOIN role_type r
    ON u.role_id = r.role_id
WHERE r.role_id = 3
    AND u.is_active = 1;
```

7.2.6 vw_Supervisor_Assigned_Maintenance_Tasks

```
CREATE OR REPLACE VIEW vw_Supervisor_Assigned_Maintenance_Tasks AS
SELECT
    mt.ticket_id,
    mt.item_id,
    i.item_code,
    i.item_name,
    i.current_status,
    s.space_id,
    s.space_code,
    s.space_name,
    s.space_type,
    mt.reporter_user_id,
    reporter.user_code AS reporter_code,
    reporter.user_name AS reporter_name,
    mt.handler_user_id AS handler_id,
    handler.user_code AS handler_code,
    handler.user_name AS handler_name,
    latest_process.process_id,
    latest_process.vendor_id,
    v.vendor_name,
    mt.reported_time,
    mt.issue_desc,
    mt.maintenance_status,
    latest_process.process_time,
    latest_process.completed_time,
    latest_process.repair_cost,
    latest_process.replaced_parts,
    latest_process.next_maintenance_date,
    latest_process.repair_result
FROM maintenance_ticket mt
JOIN item i
    ON mt.item_id = i.item_id
JOIN space s
    ON i.space_id = s.space_id
JOIN app_user reporter
    ON mt.reporter_user_id = reporter.user_id
LEFT JOIN app_user handler
    ON mt.handler_user_id = handler.user_id
LEFT JOIN (
    SELECT p1.*
    FROM maintenance_process_record p1
    JOIN (
        SELECT ticket_id, MAX(process_id) AS process_id
        FROM maintenance_process_record
        GROUP BY ticket_id
    ) latest
    ON p1.ticket_id = latest.ticket_id
```

```
        AND p1.process_id = latest.process_id
    ) latest_process
    ON mt.ticket_id = latest_process.ticket_id
LEFT JOIN vendor v
    ON latest_process.vendor_id = v.vendor_id
WHERE mt.maintenance_status IN (1, 2);
```

7.2.7 vw_Supervisor_Maintenance_History

```
CREATE OR REPLACE VIEW vw_Supervisor_Maintenance_History AS
SELECT
    mt.ticket_id,
    mt.item_id,
    i.item_code,
    i.item_name,
    i.current_status,
    s.space_id,
    s.space_code,
    s.space_name,
    s.space_type,
    mt.reporter_user_id,
    reporter.user_code AS reporter_code,
    reporter.user_name AS reporter_name,
    mt.handler_user_id AS handler_id,
    handler.user_code AS handler_code,
    handler.user_name AS handler_name,
    latest_process.process_id,
    latest_process.vendor_id,
    v.vendor_name,
    mt.reported_time,
    mt.issue_desc,
    mt.maintenance_status,
    latest_process.process_time,
    latest_process.completed_time,
    latest_process.repair_cost,
    latest_process.replaced_parts,
    latest_process.next_maintenance_date,
    latest_process.repair_result
FROM maintenance_ticket mt
JOIN item i
    ON mt.item_id = i.item_id
JOIN space s
    ON i.space_id = s.space_id
JOIN app_user reporter
    ON mt.reporter_user_id = reporter.user_id
LEFT JOIN app_user handler
    ON mt.handler_user_id = handler.user_id
LEFT JOIN (
    SELECT p1.*
    FROM maintenance_process_record p1
    JOIN (
        SELECT ticket_id, MAX(process_id) AS process_id
        FROM maintenance_process_record
        GROUP BY ticket_id
    ) latest
    ON p1.ticket_id = latest.ticket_id
```

```

        AND p1.process_id = latest.process_id
    ) latest_process
    ON mt.ticket_id = latest_process.ticket_id
LEFT JOIN vendor v
    ON latest_process.vendor_id = v.vendor_id
WHERE mt.maintenance_status IN (3, 4);

```

7.2.8 vw_Admin_Asset_Master

```

CREATE OR REPLACE VIEW vw_Admin_Asset_Master AS
SELECT
    i.item_id,
    i.item_code,
    i.item_name,
    i.item_type,
    i.current_status,
    i.warranty_expiry,
    s.space_id,
    s.space_code,
    s.space_name,
    s.space_type,
    a.asset_code,
    a.fund_source_code,
    a.fund_source_note,
    a.acquired_date,
    a.acquired_cost,
    a.useful_life_years,
    a.custodian_user_id,
    custodian.user_code AS custodian_code,
    custodian.user_name AS custodian_name,
    custodian.email AS custodian_email
FROM item i
JOIN asset_detail a
    ON i.item_id = a.item_id
JOIN space s
    ON i.space_id = s.space_id
JOIN app_user custodian
    ON a.custodian_user_id = custodian.user_id
WHERE i.item_type = 1;

```


7.2.9 vw_Admin_Consumable_Alert

```
CREATE OR REPLACE VIEW vw_Admin_Consumable_Alert AS
SELECT
    i.item_id,
    i.item_code,
    i.item_name,
    i.current_status,
    s.space_id,
    s.space_code,
    s.space_name,
    s.space_type,
    c.stock_quantity,
    c.min_stock,
    c.unit_code,
    CASE
        WHEN c.stock_quantity = 0 THEN '已無庫存'
        WHEN c.stock_quantity <= c.min_stock THEN '低於安全庫存'
        ELSE '正常'
    END AS alert_level
FROM consumable_detail c
JOIN item i
    ON c.item_id = i.item_id
JOIN space s
    ON i.space_id = s.space_id
WHERE i.item_type = 3
    AND i.current_status <> 5
    AND c.stock_quantity <= c.min_stock;
```

7.2.10 vw_Admin_Maintenance_Ticket_Master

```
CREATE OR REPLACE VIEW vw_Admin_Maintenance_Ticket_Master AS
SELECT
    mt.ticket_id,
    mt.item_id,
    i.item_code,
    i.item_name,
    i.item_type,
    i.current_status,
    s.space_id,
    s.space_code,
    s.space_name,
    s.space_type,
    mt.reporter_user_id,
    reporter.user_code AS reporter_code,
    reporter.user_name AS reporter_name,
    mt.handler_user_id AS handler_id,
    handler.user_code AS handler_code,
    handler.user_name AS handler_name,
    latest_process.process_id,
    latest_process.vendor_id,
    v.vendor_name,
    mt.reported_time,
    mt.issue_desc,
    mt.maintenance_status,
    latest_process.process_time,
    latest_process.complected_time,
    latest_process.repair_cost,
    latest_process.replaced_parts,
    latest_process.next_maintenance_date,
    latest_process.repair_result
FROM maintenance_ticket mt
JOIN item i
    ON mt.item_id = i.item_id
JOIN space s
    ON i.space_id = s.space_id
JOIN app_user reporter
    ON mt.reporter_user_id = reporter.user_id
LEFT JOIN app_user handler
    ON mt.handler_user_id = handler.user_id
LEFT JOIN (
    SELECT p1.*
    FROM maintenance_process_record p1
    JOIN (
        SELECT ticket_id, MAX(process_id) AS process_id
        FROM maintenance_process_record
        GROUP BY ticket_id
    ) latest
```

```
        ON p1.ticket_id = latest.ticket_id
        AND p1.process_id = latest.process_id
    ) latest_process
    ON mt.ticket_id = latest_process.ticket_id
LEFT JOIN vendor v
    ON latest_process.vendor_id = v.vendor_id;
```

7.2.11 vw_Admin_Audit_Trail

```
CREATE OR REPLACE VIEW vw_Admin_Audit_Trail AS
SELECT
    i.item_id AS item_id,
    i.item_code AS item_code,
    i.item_name AS item_name,
    '狀態異動' AS event_type,
    sh.changed_time AS event_time,
    op.user_id AS actor_id,
    op.user_code AS actor_code,
    op.user_name AS actor_name,
    CONCAT('狀態代碼：', sh.old_status, ' → ', sh.new_status) AS event_detail,
    sh.reason AS note
FROM status_history sh
JOIN item i
    ON sh.item_id = i.item_id
JOIN app_user op
    ON sh.operator_user_id = op.user_id

UNION ALL

SELECT
    i.item_id AS item_id,
    i.item_code AS item_code,
    i.item_name AS item_name,
    '設備借用' AS event_type,
    br.borrow_time AS event_time,
    u.user_id AS actor_id,
    u.user_code AS actor_code,
    u.user_name AS actor_name,
    CONCAT('借用狀態代碼：', br.borrow_status) AS event_detail,
    CONCAT(
        '預計歸還：',
        IFNULL(DATE_FORMAT(br.expected_return_time, '%Y-%m-%d %H:%i:%s'), '未填寫'),
        '；實際歸還：',
        IFNULL(DATE_FORMAT(br.actual_return_time, '%Y-%m-%d %H:%i:%s'), '尚未歸還')
    ) AS note
FROM borrow_record br
JOIN item i
    ON br.item_id = i.item_id
JOIN app_user u
    ON br.user_id = u.user_id

UNION ALL

SELECT
    i.item_id AS item_id,
    i.item_code AS item_code,
```

```

i.item_name AS item_name,
'維修報修' AS event_type,
mt.reported_time AS event_time,
reporter.user_id AS actor_id,
reporter.user_code AS actor_code,
reporter.user_name AS actor_name,
CONCAT('維修狀態代碼：', mt.maintenance_status) AS event_detail,
mt.issue_desc AS note
FROM maintenance_ticket mt
JOIN item i
    ON mt.item_id = i.item_id
JOIN app_user reporter
    ON mt.reporter_user_id = reporter.user_id

UNION ALL

SELECT
i.item_id AS item_id,
i.item_code AS item_code,
i.item_name AS item_name,
'維修處理' AS event_type,
COALESCE(mpr.completed_time, mpr.process_time) AS event_time,
handler.user_id AS actor_id,
handler.user_code AS actor_code,
handler.user_name AS actor_name,
CONCAT(
    '維修費用：',
    IFNULL(CAST(mpr.repair_cost AS CHAR), '未填寫'),
    '；廠商：',
    IFNULL(v.vendor_name, '系內處理'),
    '；維修結果：',
    IFNULL(mpr.repair_result, '未填寫')
) AS event_detail,
CONCAT(
    '完成時間：',
    IFNULL(DATE_FORMAT(mpr.completed_time, '%Y-%m-%d %H:%i:%s'), '尚未完成'),
    '；更換零件：',
    IFNULL(mpr.replaced_parts, '無'),
    '；下次保養：',
    IFNULL(DATE_FORMAT(mpr.next_maintenance_date, '%Y-%m-%d'), '未設定')
) AS note
FROM maintenance_process_record mpr
JOIN maintenance_ticket mt
    ON mpr.ticket_id = mt.ticket_id
JOIN item i
    ON mt.item_id = i.item_id
LEFT JOIN app_user handler
    ON mt.handler_user_id = handler.user_id

```

```
LEFT JOIN vendor v  
ON mpr.vendor_id = v.vendor_id;
```

7.3 equipment_web 直接 SQL 查詢頁面

7.1 已整理 equipment_web 目前查詢使用的 View。另有以下管理頁面目前使用直接 SQL 查詢，並未透過 View：

系統使用位置	查詢方式	說明
/admin/items	直接 SQL	管理員完整物品清單，可依物品類型與狀態篩選。
/admin/consumables	直接 SQL	管理員耗材清單，顯示庫存、安全庫存與庫存警示資訊。